



<https://erqos.com/>

PLC + IoT ด้วย AI โดยใช้ Industrial ESP32
(EP.9) MQTT Protocol

JSON (JavaScript Object Notation)



- JSON (JavaScript Object Notation) เป็นรูปแบบข้อมูลที่ ใช้ในการแลกเปลี่ยนข้อมูลระหว่างระบบ อย่างง่ายและมีความเข้าใจง่าย โดยมีโครงสร้างที่อ่านง่ายทั้งสำหรับมนุษย์และคอมพิวเตอร์ ใช้ใน หลายภาษาโปรแกรมและแอปพลิเคชัน { "ชื่อของข้อมูล (Key)" : "ข้อมูล (Value)" }

```
{  
  "name_name" : "Machine_1",  
  "machine_status" : true,  
  "work_status" : false,  
  "actual" : 100  
}
```

การทำงานของโปรโตคอล MQTT



MQTT ประกอบด้วย 2 ส่วน คือ

- **ส่วนที่ 1 - MQTT Client**

เป็นส่วน publish ข้อมูลต่างๆ ขึ้นไปยัง MQTT Broker และสามารถ Subscribe ข้อมูลต่างๆจาก MQTT Broker ผ่านทาง TCP/IP Protocol ถ้ามองในมุมมองของ Internet of Things (IoT) อุปกรณ์จะเป็น Device ที่สามารถเชื่อมต่อกับระบบเครือข่ายได้ เช่น บอร์ด ESP32, บอร์ด Raspberry Pi, เว็บไซต์, สมาร์ทโฟน

การทำงานของโปรโตคอล MQTT



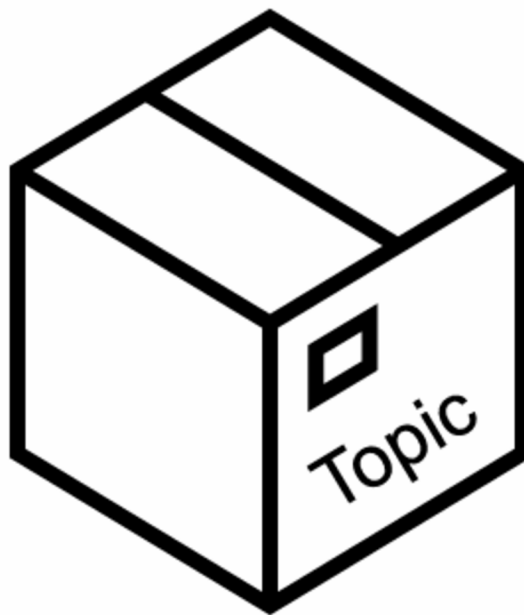
- ส่วนที่ 2 - MQTT Broker หรือ MQTT Server

เป็นซอฟต์แวร์สำหรับรับข้อมูลจาก MQTT Client ที่ได้ publish เข้ามาและสามารถ publish ข้อมูลจาก MQTT Broker ไปยัง MQTT Client ที่ได้ Subscribe ข้อมูลไว้ หากมองในมุมมองของ Internet of Things อาจจะเป็น Cloud Server ของค่ายต่างๆ เช่น CloudMQTT, NETPIE, Azure, AWS เป็นต้น หรือใช้ Single Board Computer เช่นบอร์ด Raspberry Pi, LattePanda, Beagle Bone, nanoPi, อื่นๆ แล้ว ติดตั้งซอฟต์แวร์เพิ่มเติมก็สามารถใช้งานได้เช่นกัน



Topic

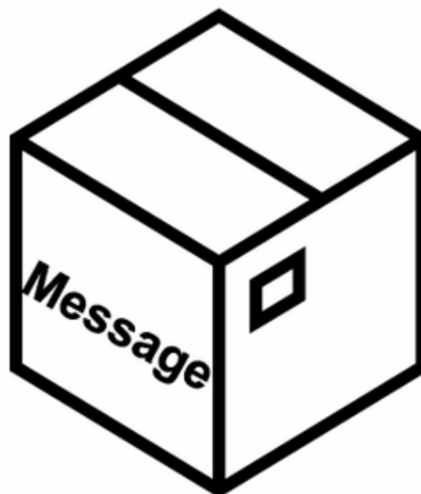
เป็นตัวกำหนดหมวดหมู่หรือช่องทางที่ใช้สำหรับการจัดการการส่งข้อความ





Message

- เป็นข้อมูลที่ส่งผ่าน Topic ซึ่งมีองค์ประกอบหลัก:
 - **Payload:** ข้อมูลจริง เช่น ค่าอุณหภูมิ, ข้อความ JSON
 - **Properties:** ส่วนเพิ่มเติม เช่น QoS, Retain Flag, หรือ Timestamp



การทำงานของโปรโตคอล MQTT



Topic and Message

Machine_001_data

```
{  
  "name_name" : "Machine_001",  
  "machine_status" : true,  
  "work_status" : false,  
  "actual" : 100  
  "ng" : 1  
}, {"qos":0, "Timestamp":___}
```

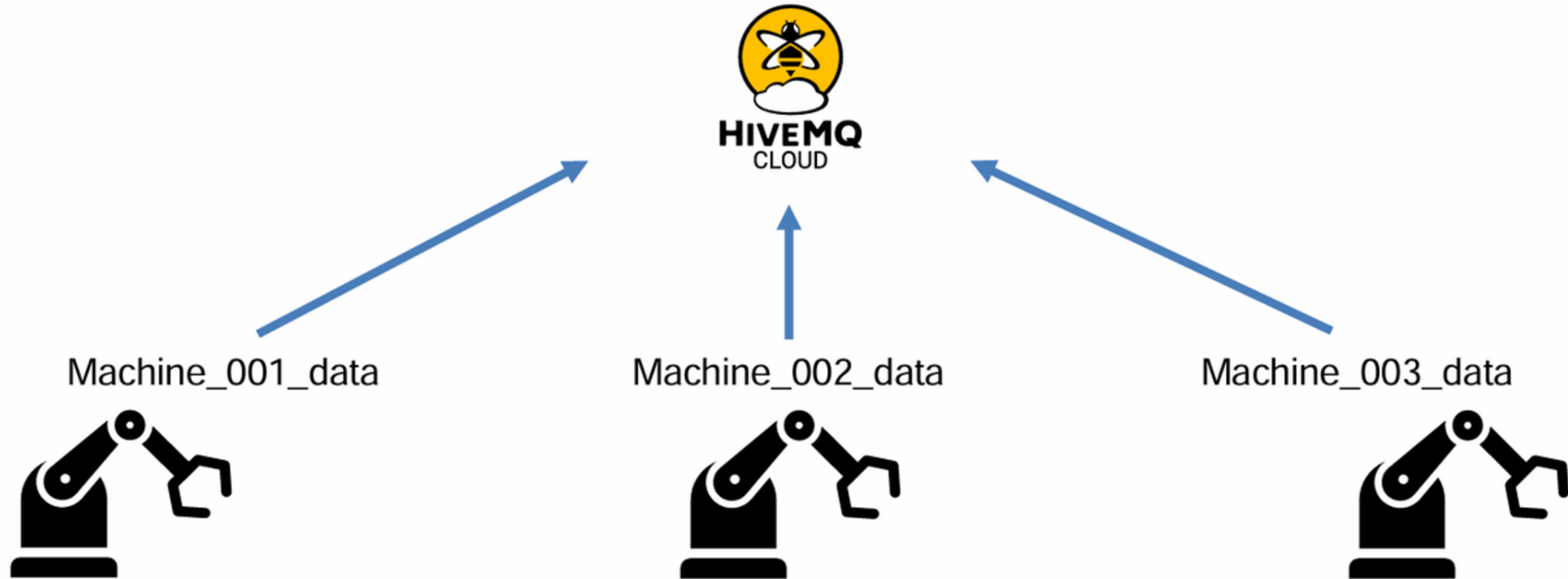
Machine_002_data

```
{  
  "name_name" : "Machine_002",  
  "machine_status" : true,  
  "work_status" : false,  
  "actual" : 100  
  "ng" : 1  
}, {"qos":0, "Timestamp":___}
```

Machine_003_data

```
{  
  "name_name" : "Machine_003",  
  "machine_status" : true,  
  "work_status" : false,  
  "actual" : 100  
  "ng" : 1  
}, {"qos":0, "Timestamp":___}
```

การทำงานของโปรโตคอล MQTT



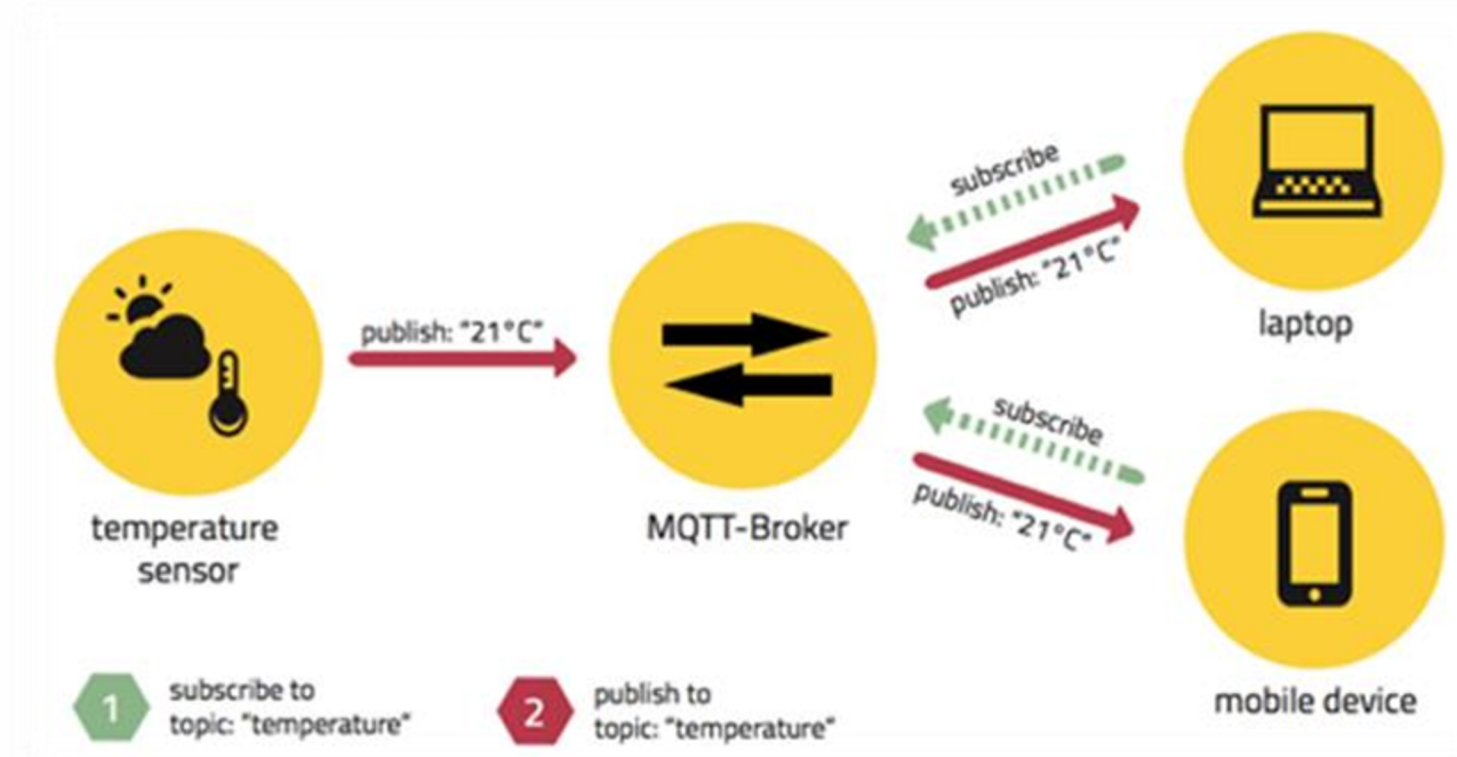
การทำงานของโปรโตคอล MQTT



- MQTT มีองค์ประกอบหลักอยู่ 2 ส่วนคือ

1. Client

2. Server



การทำงานของโปรโตคอล MQTT



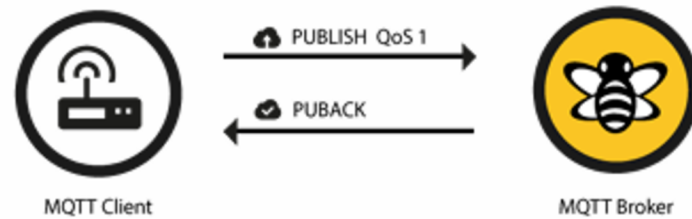
- **Quality of Service**

- QoS is a major feature of MQTT, it makes communication in unreliable networks a lot easier because the protocol handles **retransmission** and guarantees the delivery of the message
- The Quality of Service (**QoS**) level is an agreement between sender and receiver of a message regarding the guarantees of delivering a message.
 - There are 3 QoS levels in MQTT:
 - At most once (0)
 - At least once (1)
 - Exactly once (2)

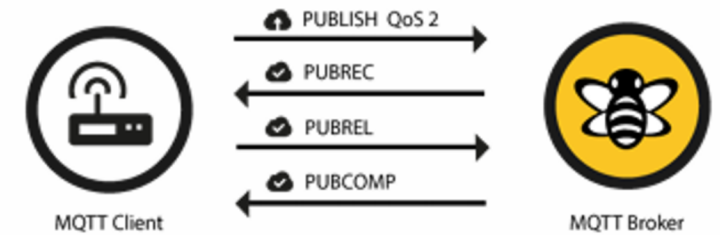
การทำงานของโปรโตคอล MQTT



QoS 0 : fire and forget



QoS 1 : If the PUBACK isn't received in a reasonable amount of time the sender will resend the PUBLISH message



QoS 2 : When the flow is completed both parties can be sure that the message has been delivered and the sender also knows about it.

Whenever a packet gets lost on the way, **the sender is responsible** for resending the last message after a reasonable amount of time.

การทำงานของโปรโตคอล MQTT



QoS 0

- QoS0 ส่งข้อมูลเพียงครั้งเดียว ปลายทางได้รับหรือไม่ได้รับก็ได้



การทำงานของโปรโตคอล MQTT



QoS 1

- QoS1 ส่งข้อมูลเพียงครั้งเดียว ปลายทางได้รับหรือไม่ได้รับก็ได้ และ ให้จำข้อมูลการส่งครั้งล่าสุดไว้เสมอ หากมีการเชื่อมต่อเข้ามา จะได้รับข้อความสุดท้ายอีกครั้ง

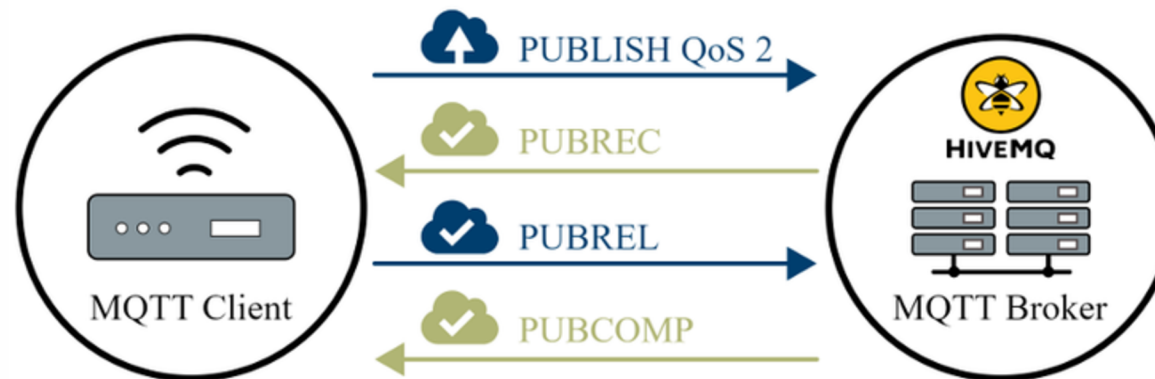


การทำงานของโปรโตคอล MQTT



QoS 2

- QoS2 ส่งข้อมูลจนกว่าปลายทางจะได้รับข้อมูล ซึ่งมีการเช็คตลอดเวลา มีความแม่นยำสูงแต่ทำงานช้ากว่า QoS0 และ QoS1



การทำงานของโปรโตคอล MQTT



- **Retained messages**

- The broker will store the last retained message and the corresponding QoS for that topic.
- Each client that subscribes to a topic pattern, which matches the topic of the retained message, will receive the message immediately **after subscribing**.
- For each topic only one retained message will be stored by the broker.
- A retained message makes sense, when newly connected subscribers should receive messages immediately and shouldn't have to wait until a publishing client sends the next message.

- **Retain Message** ใน MQTT คือ ข้อความที่ **Broker** เก็บไว้สำหรับหัวข้อ (Topic) หนึ่ง ๆ เพื่อให้ **Client** ที่เชื่อมต่อใหม่และ **Subscribe** หัวข้อนั้นสามารถรับข้อมูลล่าสุดได้ทันที
หน้าที่ของ **Retain Message** :
 - ให้ข้อมูลล่าสุดแก่ **Subscriber** ใหม่โดยไม่ต้องรอ **Publisher** ส่งข้อมูลอีกครั้ง
 - ลดการรอคอยข้อมูลครั้งแรก โดยเฉพาะในระบบที่ข้อมูลถูกส่งมาไม่บ่อย

ตัวอย่างการใช้งานของ MQTT

